

CoreMonitor

Plano de Testes Manuais e Automacao de API Compliance & Quality Assurance - Sprint 4

Projeto	CoreMonitor
Curso	FIAP - Analise e Desenvolvimento de Sistemas
Materia	Compliance & Quality Assurance
Equipe	Igor Neris Soares Alves - RM560088 Guilherme Jun Conheci - RM559986
Azure Boards	https://dev.azure.com/OrgCoreMonitor/CoreMonitor
Ferramenta de automacao	Bruno API Client

Documento elaborado para registrar o plano de testes manuais em nivel de sistema e a estrategia de testes automatizados da aplicacao CoreMonitor, considerando a indisponibilidade do recurso completo Azure Test Plans no ambiente da equipe.

1. Objetivo do documento

Este documento apresenta o plano de testes manuais do CoreMonitor, contemplando as funcionalidades principais da aplicacao, os dados controlados de entrada, os resultados esperados e o procedimento de execucao de cada teste. Tambem apresenta a estrategia de automacao de testes de API utilizando o Bruno API Client, conforme solicitado para a etapa de Compliance & Quality Assurance.

O plano foi estruturado para atender a Parte A da entrega, enquanto os testes automatizados descritos neste documento servem como base para a Parte B, cuja execucao deve ser demonstrada em video.

2. Visao geral da solucao testada

O CoreMonitor e uma plataforma contábil-financeira integrada a uma proposta de IoT e analise inteligente de dados. A aplicacao organiza informacoes financeiras, como centros de custo, contas contabeis, registros contabeis, clientes, vendas e compras, formando uma base operacional para consultas, relatorios, dashboards e futuras analises preditivas.

No contexto desta entrega de CQA, o escopo de validacao esta concentrado nas funcionalidades transacionais do sistema web e nos endpoints correspondentes da API, considerando os fluxos de cadastro, consulta, validacao de dados obrigatorios e tratamento de entradas invalidas.

Link de acompanhamento no Azure Boards

<https://dev.azure.com/OrgCoreMonitor/CoreMonitor>

3. Escopo dos testes

- Validar as funcionalidades principais do sistema CoreMonitor em nivel de sistema.
- Cobrir cadastros e consultas de centros de custo, contas contabeis, registros contabeis, clientes, vendas e compras.
- Utilizar dados controlados, com entrada e saida esperada definidas previamente.
- Registrar cenarios positivos e negativos para demonstrar funcionamento correto e tratamento de falhas.
- Complementar a validacao manual com automacao de API utilizando Bruno.

4. Ambiente de testes

Item	Descrição	Observação
Aplicacao web	CoreMonitor em ambiente local	http://localhost:8081
API	Endpoints REST do CoreMonitor	URL base a configurar no Bruno conforme ambiente da equipe
Banco de dados	Base relacional do projeto	Tabelas financeiras/contabeis usadas pelo CoreMonitor
Ferramenta manual	Navegador web	Google Chrome ou equivalente
Ferramenta automatizada	Bruno API Client	Colecao de testes exportada para o repositorio GitHub

5. Evidencias visuais do sistema utilizado como base

As telas abaixo demonstram os principais modulos considerados no plano de testes. Elas representam a camada operacional do CoreMonitor utilizada para cadastro, consulta e validacao dos dados financeiros e contabeis.

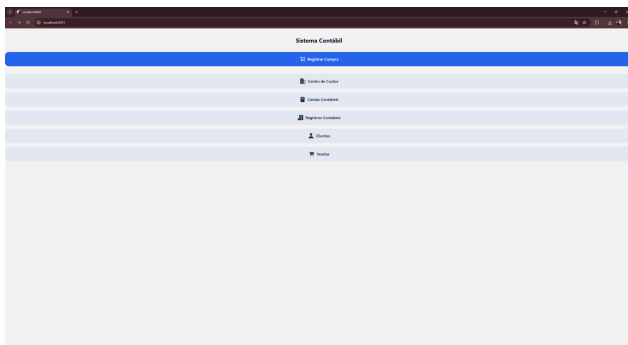


Figura 1 - Tela inicial do Sistema Contabil

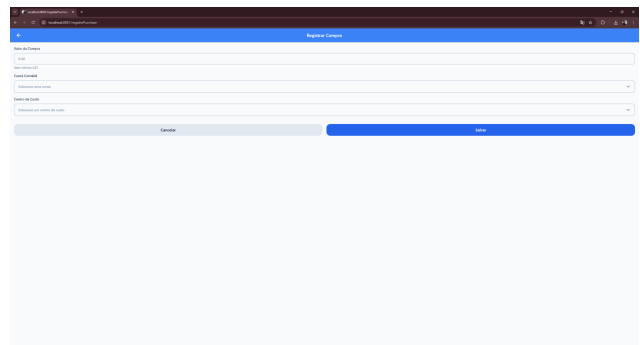


Figura 2 - Tela de registro de compra

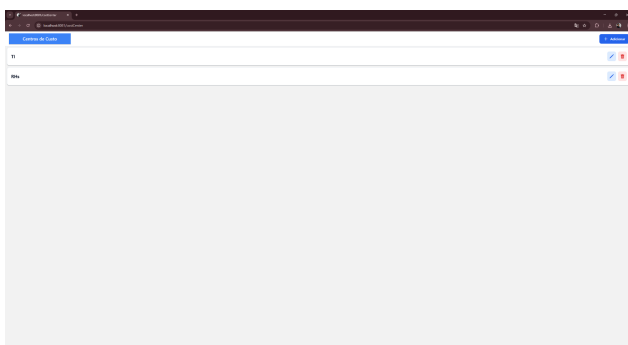


Figura 3 - Tela de centros de custo

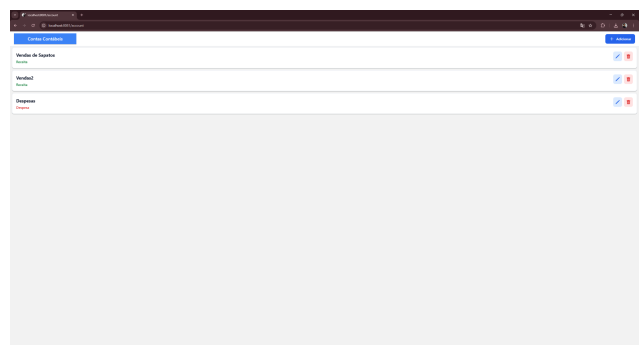


Figura 4 - Tela de contas contabeis

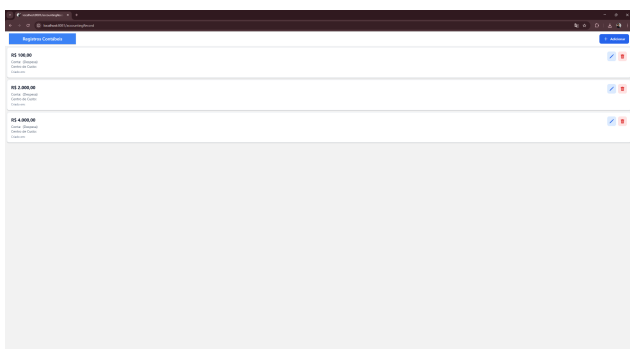


Figura 5 - Tela de registros contabeis

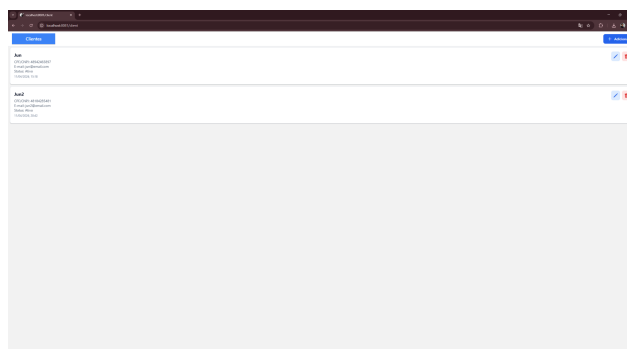


Figura 6 - Tela de clientes

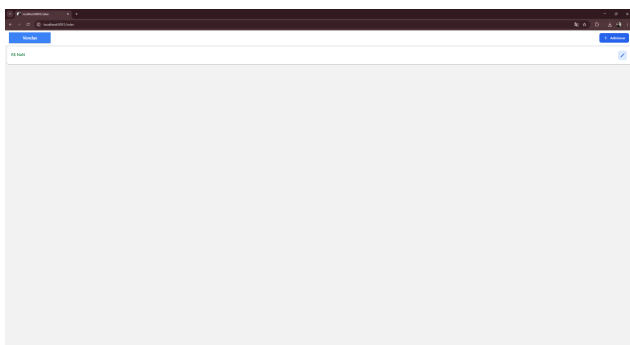


Figura 7 - Tela de vendas

Observacao: as evidencias visuais foram utilizadas para alinhar os casos de teste com os modulos existentes no prototipo do CoreMonitor.

6. Estrategia e criterios de teste

A estrategia adotada contempla testes manuais de validacao em nivel de sistema e testes automatizados de API. Os testes manuais verificam se o comportamento do sistema esta coerente com o fluxo esperado pelo usuario. Ja os testes automatizados validam respostas da API em cenarios controlados, incluindo um fluxo positivo e tres fluxos negativos.

Criterios gerais de aceite

- O sistema deve permitir o cadastro de dados validos nas funcionalidades principais.
- O sistema deve apresentar dados cadastrados nas respectivas listagens.
- O sistema deve bloquear entradas invalidas, vazias ou inconsistentes.
- A API deve retornar status coerente com o resultado esperado de cada requisicao.
- Os testes automatizados devem demonstrar pelo menos um caso de sucesso e tres casos de erro controlado.

7. Resumo dos casos de teste manuais

Codigo	Funcionalidade	Cenario	Saída esperada
CT01	Centro de custo	Cadastro valido	Centro de custo exibido na listagem
CT02	Conta contabil	Cadastro de receita valida	Conta exibida com tipo Receita
CT03	Conta contabil	Cadastro de despesa valida	Conta exibida com tipo Despesa
CT04	Cliente	Cadastro valido	Cliente exibido na listagem
CT05	Registro contabil	Cadastro valido	Registro exibido com valor, conta e centro de custo
CT06	Compra	Registro valido	Compra salva/classificada corretamente
CT07	Centro de custo	Nome vazio	Sistema bloqueia o cadastro
CT08	Cliente	E-mail invalido	Sistema bloqueia ou sinaliza inconsistencia
CT09	Registro contabil	Valor negativo	Sistema rejeita o valor invalido
CT10	Consulta/API	ID inexistente	Sistema retorna mensagem de nao encontrado

8. Casos de teste manuais - Parte A

CT01 - Cadastro de centro de custo valido	Status esperado: Planejado
Objetivo	Validar se o usuario consegue cadastrar um centro de custo com nome valido.
Dados de entrada	Nome do centro de custo: QA - Teste
Dados de saida esperados	O centro de custo QA - Teste deve ser salvo e exibido na tela de centros de custo.
Procedimento de teste	1. Acessar a tela inicial do CoreMonitor. 2. Entrar no modulo Centros de Custo. 3. Clicar em Adicionar. 4. Informar o nome QA - Teste. 5. Salvar o cadastro. 6. Verificar se o novo centro de custo aparece na listagem.
CT02 - Cadastro de conta contabil de receita	Status esperado: Planejado
Objetivo	Validar se o sistema permite cadastrar uma conta contabil classificada como receita.
Dados de entrada	Nome da conta: Receita QA; Tipo: Receita
Dados de saida esperados	A conta Receita QA deve ser exibida na listagem com classificacao Receita.
Procedimento de teste	1. Acessar a tela Contas Contabeis. 2. Clicar em Adicionar. 3. Preencher o nome Receita QA. 4. Selecionar o tipo Receita. 5. Salvar. 6. Conferir a exibicao da conta na listagem.
CT03 - Cadastro de conta contabil de despesa	Status esperado: Planejado
Objetivo	Validar se o sistema permite cadastrar uma conta contabil classificada como despesa.
Dados de entrada	Nome da conta: Despesa QA; Tipo: Despesa
Dados de saida esperados	A conta Despesa QA deve ser exibida na listagem com classificacao Despesa.
Procedimento de teste	1. Acessar Contas Contabeis. 2. Clicar em Adicionar. 3. Preencher o nome Despesa QA. 4. Selecionar o tipo Despesa. 5. Salvar. 6. Verificar se a conta foi apresentada corretamente.
CT04 - Cadastro de cliente valido	Status esperado: Planejado
Objetivo	Validar o cadastro de um cliente com dados obrigatorios preenchidos corretamente.
Dados de entrada	Nome: Cliente QA; CPF/CNPJ: 12345678901; E-mail: cliente.qa@teste.com; Status: Ativo
Dados de saida esperados	O cliente deve ser salvo e exibido na tela Clientes com os dados informados.
Procedimento de teste	1. Acessar a tela Clientes. 2. Clicar em Adicionar. 3. Preencher nome, CPF/CNPJ, e-mail e status. 4. Salvar o cadastro. 5. Verificar se o cliente aparece na listagem.

CT05 - Cadastro de registro contabil valido	Status esperado: Planejado
Objetivo	Validar se o sistema permite registrar um lancamento contabil com valor, conta e centro de custo.
Dados de entrada	Valor: 100,00; Conta: Despesa QA; Centro de custo: QA - Teste
Dados de saida esperados	O registro contabil deve ser exibido com valor R\$ 100,00 e vinculos preenchidos.
Procedimento de teste	1. Acessar Registros Contabeis. 2. Clicar em Adicionar. 3. Informar o valor 100,00. 4. Selecionar a conta contabil. 5. Selecionar o centro de custo. 6. Salvar e verificar a listagem.

CT06 - Registro de compra valido	Status esperado: Planejado
Objetivo	Validar o fluxo de registro de compra com classificacao contabil e centro de custo.
Dados de entrada	Valor da compra: 250,50; Conta contabil: Despesa QA; Centro de custo: QA - Teste
Dados de saida esperados	A compra deve ser salva sem erro e vinculada a uma conta contabil e centro de custo.
Procedimento de teste	1. Acessar Registrar Compra. 2. Informar valor 250,50. 3. Selecionar a conta contabil. 4. Selecionar o centro de custo. 5. Clicar em Salvar. 6. Confirmar que o sistema aceita a operacao.

CT07 - Cadastro de centro de custo com nome vazio	Status esperado: Planejado
Objetivo	Validar o tratamento de erro para tentativa de cadastrar centro de custo sem nome.
Dados de entrada	Nome do centro de custo: vazio
Dados de saida esperados	O sistema deve impedir o cadastro e sinalizar que o campo nome e obrigatorio.
Procedimento de teste	1. Acessar Centros de Custo. 2. Clicar em Adicionar. 3. Deixar o campo nome vazio. 4. Clicar em Salvar. 5. Verificar se o cadastro foi bloqueado.

CT08 - Cadastro de cliente com e-mail invalido	Status esperado: Planejado
Objetivo	Validar se o sistema trata dados cadastrais inconsistentes.
Dados de entrada	Nome: Cliente Erro; CPF/CNPJ: 12345678902; E-mail: email-invalido
Dados de saida esperados	O sistema deve bloquear o cadastro ou indicar inconsistencia no e-mail.
Procedimento de teste	1. Acessar Clientes. 2. Clicar em Adicionar. 3. Preencher os dados do cliente com e-mail invalido. 4. Clicar em Salvar. 5. Verificar se o sistema impede ou sinaliza o erro.

CT09 - Registro contabil com valor negativo	Status esperado: Planejado
Objetivo	Validar se o sistema rejeita lancamentos financeiros com valor menor que zero.
Dados de entrada	Valor: -100,00; Conta: Despesa QA; Centro de custo: QA - Teste
Dados de saida esperados	O sistema deve rejeitar o registro e informar que o valor deve ser maior que zero.
Procedimento de teste	1. Acessar Registros Contabeis. 2. Clicar em Adicionar. 3. Informar valor negativo. 4. Selecionar conta e centro de custo. 5. Tentar salvar. 6. Verificar se o sistema bloqueia o cadastro.

CT10 - Consulta de registro inexistente	Status esperado: Planejado
Objetivo	Validar o comportamento do sistema/API ao consultar um identificador inexistente.
Dados de entrada	ID consultado: 99999
Dados de saida esperados	O sistema/API deve retornar que o registro nao foi encontrado.
Procedimento de teste	1. Executar a consulta na interface ou API. 2. Informar o ID 99999. 3. Enviar a requisicao. 4. Verificar se o retorno indica registro inexistente.

9. Testes automatizados com Bruno - Parte B

Para a automacao de testes, sera utilizado o Bruno API Client, ferramenta equivalente para organizacao e execucao de requisicoes HTTP. A colecao de testes deve ser exportada e adicionada ao repositorio GitHub da entrega.

A estrategia definida utiliza quatro cenarios automatizados: um cenario positivo, comprovando o funcionamento correto da API, e tres cenarios negativos, validando tratamento de erro com dados controlados.

	Tipo	Requisicao	Entrada controlada	Resposta esperada
	Sucesso	POST /clientes	Cliente valido	201 Created ou 200 OK
	Erro	POST /clientes	Nome vazio ou campo obrigatorio ausente	400 Bad Request
	Erro	POST /registros-contabeis	Valor negativo	400 Bad Request
	Erro	GET /clientes/99999	ID inexistente	404 Not Found

Detalhamento dos testes automatizados

TA01 - Cadastro valido de cliente via API	Status esperado: Automatizado no Bruno
Objetivo	Validar se a API cadastra um cliente quando todos os dados obrigatorios sao enviados corretamente.
Dados de entrada	<pre>{ "nome": "Cliente Bruno QA", "cpfCnpj": "12345678901", "email": "cliente.bruno@teste.com", "ativo": true }</pre>
Dados de saida esperados	Status HTTP 200 OK ou 201 Created; corpo de resposta indicando cadastro realizado com sucesso.
Procedimento de teste	1. Abrir o Bruno. 2. Selecionar a colecao CoreMonitor. 3. Executar a requisicao POST de cliente valido. 4. Conferir o status HTTP. 5. Verificar se a resposta confirma o cadastro.

TA02 - Cadastro de cliente com nome vazio	Status esperado: Automatizado no Bruno
Objetivo	Validar se a API rejeita cadastro sem um campo obrigatorio.
Dados de entrada	<pre>{ "nome": "", "cpfCnpj": "12345678901", "email": "cliente.erro@teste.com", "ativo": true }</pre>
Dados de saida esperados	Status HTTP 400 Bad Request; corpo de resposta com mensagem de validacao.
Procedimento de teste	1. Abrir a requisicao negativa no Bruno. 2. Enviar payload com nome vazio. 3. Executar a requisicao. 4. Conferir o status 400. 5. Verificar a mensagem de erro esperada.

TA03 - Registro contabil com valor negativo via API	Status esperado: Automatizado no Bruno
Objetivo	Validar se a API rejeita lancamento contabil com valor invalido.
Dados de entrada	{ "valor": -100, "contald": 1, "centroCustold": 1 }
Dados de saida esperados	Status HTTP 400 Bad Request; resposta informando que o valor deve ser maior que zero.
Procedimento de teste	1. Abrir a requisicao de registro contabil invalido. 2. Enviar payload com valor negativo. 3. Executar a requisicao. 4. Conferir o status HTTP. 5. Validar a mensagem de erro.

TA04 - Consulta de cliente inexistente via API	Status esperado: Automatizado no Bruno
Objetivo	Validar se a API retorna erro controlado quando o cliente nao existe.
Dados de entrada	GET /clientes/99999
Dados de saida esperados	Status HTTP 404 Not Found; resposta informando cliente ou registro nao encontrado.
Procedimento de teste	1. Abrir a requisicao GET no Bruno. 2. Informar o ID 99999 na rota. 3. Executar a requisicao. 4. Conferir o status 404. 5. Validar o retorno de nao encontrado.

Observacao sobre endpoints

Os caminhos dos endpoints devem ser ajustados conforme a nomenclatura real da API do CoreMonitor. O objetivo dos testes permanece o mesmo: validar um fluxo de sucesso e tres fluxos negativos controlados.

10. Organizacao sugerida do repositorio GitHub

O PDF pode ser adicionado diretamente ao repositorio GitHub da Sprint 4, preferencialmente na branch develop, junto com a colecao de testes do Bruno e evidencias da execucao.

```
coremonitor-cqa-sprint4/  
|-- README.md  
|-- docs/  
| |-- plano-testes-manuais-coremonitor.pdf  
|-- bruno/  
| |-- coremonitor-api-tests/ ou coremonitor-tests.json  
|-- evidencias/  
| |-- prints-sistema/  
| |-- execucao-bruno.png  
|-- links/  
| |-- links-entrega.txt
```

11. Checklist de entrega

Item	Descricao	Status
PDF do plano de testes	Documento com testes manuais, entradas, saidas esperadas e procedimentos.	Gerado
Testes automatizados	Colecao do Bruno com ao menos quatro cenarios.	A executar/exportar
Video demonstrativo	Gravacao mostrando configuracao e execucao dos testes no Bruno.	A gravar
GitHub publico	Repositorio com PDF, colecao do Bruno, README e evidencias.	A publicar
Azure Boards	Link do projeto CoreMonitor para apoio e rastreabilidade.	Disponivel

12. Conclusao

O plano de testes definido para o CoreMonitor cobre as funcionalidades centrais do sistema e organiza os cenarios de validacao com dados controlados, procedimentos claros e resultados esperados. A combinacao entre testes manuais documentados e testes automatizados no Bruno fortalece a confiabilidade da entrega e demonstra a aplicacao pratica dos conceitos de Compliance & Quality Assurance.